Lecture 6: From Gradients to Neural Networks

Ivy Yang

Purpose and connection to Lectures 1–5

The earlier lectures established forecasting questions, honest evaluation, linear and regularized regression, trees and Random Forest, uncertainty and decision costs. This final lecture develops a conceptual bridge: vectors describe several quantities together; gradients describe how loss changes; gradient-based learning improves a rule; boosting adds correction functions; support-vector methods use geometry and representations; neural networks compose learned transformations; LSTM extends a neural network to ordered observations.

The learning goal is a visual understanding of how these methods work and connect to neural networks. Read each concept and its picture first. Detailed calculation references can be used later alongside the Coding Note.

Every new term is defined before it is used to build the next idea. Examples 6.1–6.15 and added Example 6.8A have complete executable code, generated figures and work-reference paragraphs in the Coding Note. All datasets and fixed network parameters in this chapter are labeled illustrations. The new methods introduce no additional homework or project requirements. The project reminder remains at the end.

Contents

1 Vectors, Parameters and the Gradient 2
1.1 A vector keeps several numbers in a stated order 2
1.2 A gradient describes local changes in loss 4

2 Learning by Gradient-Based Updates 5
2.1 Gradient descent takes a scaled downhill step 5
2.2 Learning rate changes the size of the update 6

3 Gradient Boosting: Correcting a Prediction Rule 7
3.1 Boosting adds a correction function 7
3.2 Why the correction follows a negative gradient 8

4 Support Vectors, Margins and Nonlinear Representations 9
4.1 SVM learns a separating boundary and a margin 9
4.2 A feature map can make a curved boundary simpler 11
4.3 Gaussian kernel: nearby points have more influence 12

4.4 SVR applies support-vector ideas to a numerical target 14

5 Neural Networks: Layers, Transformations and Learning 15
5.1 One neural-network unit computes a weighted transformation 15
5.2 Layers compose transformations: a folding analogy 16
5.3 Backpropagation computes gradients through the layers 17

6 Ordered Inputs and LSTM Memory 18
6.1 Sequence order can matter to a prediction 18
6.2 LSTM controls retained, incoming and output information 19
6.3 Prepare a time-series window for a sequence model 21

7 Begin and Complete Your Selected Final Project 22
7.1 Choose one of the three existing cases 22
7.2 Complete the tasks in the selected notebook 22
7.3 Deadline and Blackboard submission 22
7.4 Completion credit and correct-step credit 22

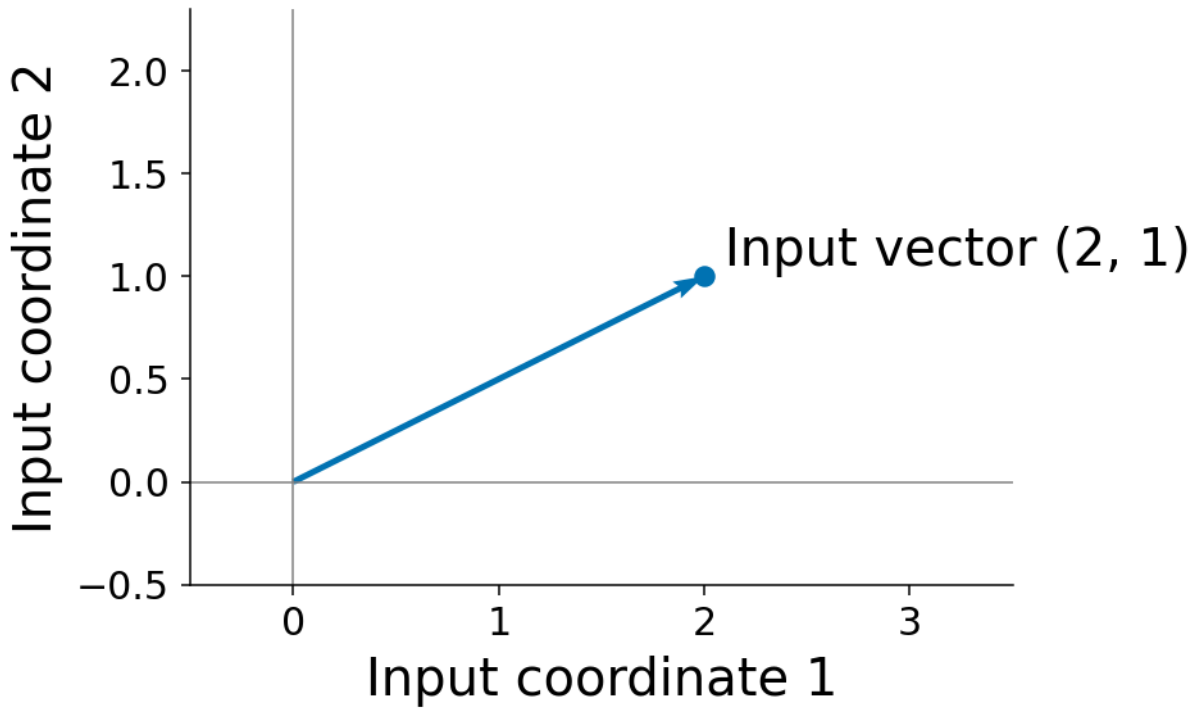
1 Vectors, Parameters and the Gradient

1.1 A vector keeps several numbers in a stated order

Definition: The idea

A scalar is one number. A vector is an ordered list of numbers. Each position is a coordinate; the number of positions is its dimension. A two-coordinate vector can be drawn as an arrow from the origin. A dot product multiplies matching coordinates and adds the products.

Follow the picture. Imagine a shop records today's customer count and average spending. These are two different measurements kept in a fixed order. A vector is the list holding them. The two axes in our small picture simply give each list position its own direction.



Coding Note, Example 6.1.

Where to use the idea. Refer here when checking the order of columns in a model input or interpreting a weighted score. Keep variable names and units with the array.

1.1.1 Example 6.1: optional calculation reference

$$\mathbf{x} = (x_1, x_2), \qquad \mathbf{w} \cdot \mathbf{x} = w_1 x_1 + w_2 x_2$$

1. Construct an input and a weight vector. Use two standardized illustrative inputs, x=(2,1), and weights w=(3,-1). The weighted sum is 5; reordering only the inputs changes its meaning. Coding Note 6.1.1.

2. Draw the labeled input arrow. Use fixed axes with one unit per coordinate. The arrow endpoint contains the two values in order. Coding Note 6.1.2.

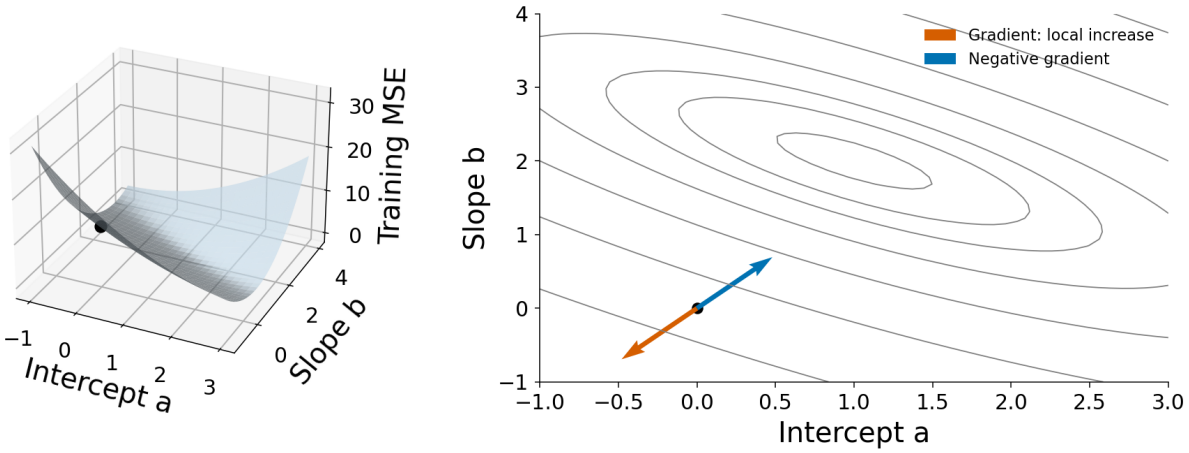
The ordered input is (2,1); the ordered weights are (3,-1). Multiply the first pair, then the second, then add: 3 times 2 plus -1 times 1 equals 5. The same arithmetic underlies a linear-model score and the weighted input to a neural-network unit.

1.2 A gradient describes local changes in loss

Definition: The idea

A gradient describes how prediction error responds to small changes in model settings. It helps choose a direction for the next adjustment.

Follow the picture. Imagine two knobs controlling a fitted line. One raises the whole line; the other tilts it. We want settings that give smaller prediction errors. Now draw the intercept on one floor axis and the slope on the other. Put loss on the vertical axis. Loss is our numerical summary of prediction error. Every point on this surface chooses a line and shows its error. Low positions mean smaller errors. Our starting line predicts zero for every input, so it misses the observed values. Moving around this bowl means trying different settings for the same model. The observations stay fixed while the settings change.



Coding Note, Example 6.2.

Where to use the idea. Refer here to distinguish model parameters from input features, inspect a gradient sign or check whether an optimizer is moving uphill or downhill.

1.2.1 Example 6.2: optional calculation reference

$$J(a,b) = \frac{1}{3}\sum_{i=1}^{3}(a + bx_i - y_i)^2, \qquad \nabla J = (J_a, J_b)$$

1. Define a line and its training loss. Use x=(0,1,2), y=(1,3,5), and initial intercept and slope zero. The zero prediction line has mean squared error 11.6667. Coding Note 6.2.1.

2. Check the local slopes numerically. Perturb one parameter at a time by a small amount. Finite differences agree with the two computed gradient coordinates. Coding Note 6.2.2.

3. Draw the loss bowl and opposite arrows. Evaluate the same loss on a fixed parameter grid. The gradient arrow rises locally; its opposite points toward a local decrease. Coding Note 6.2.3.

For the three pairs (0,1), (1,3), (2,5), start at a=0 and b=0. Predictions are all zero. Squared misses are 1,9,25; their average is 35/3. Increasing a slightly reduces loss at this point, so its partial derivative is negative. Increasing b also reduces loss. The two gradient coordinates are -6 and -8.6667. The vector records local rates of loss change in a specified parameter coordinate system.

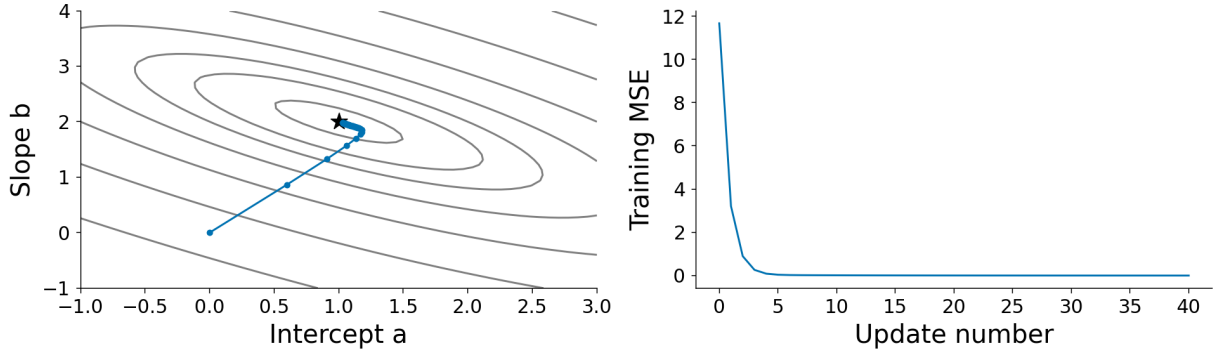
2 Learning by Gradient-Based Updates

2.1 Gradient descent takes a scaled downhill step

Definition: The idea

Gradient descent repeatedly uses the local downhill direction to adjust a model by small steps.

Follow the picture. Start with the current intercept and slope. Calculate their gradient to find a local direction. The learning rate controls how much we change the settings on this move. Follow the blue point as it makes a small downhill step. Its position changes the line, and its lower height means smaller training error. Recalculate the gradient at the new position before taking another step. Repeating this process is gradient descent. One step is one training update. In this small example, the path approaches settings that fit the three observed points. A useful downhill direction and a suitable step size work together.



Coding Note, Example 6.3.

Where to use the idea. Refer here when reading an iterative training loop or diagnosing the sign and scale of a parameter update. Report training and later evaluation loss separately.

2.1.1 Example 6.3: optional calculation reference

$$\boldsymbol{\theta}_{k+1} = \boldsymbol{\theta}_k - \eta \nabla J(\boldsymbol{\theta}_k)$$

1. Compute one explicit update. Start from the previous line and use learning rate .1. Subtracting the negative gradient coordinates increases both coefficients. Coding Note 6.3.1.

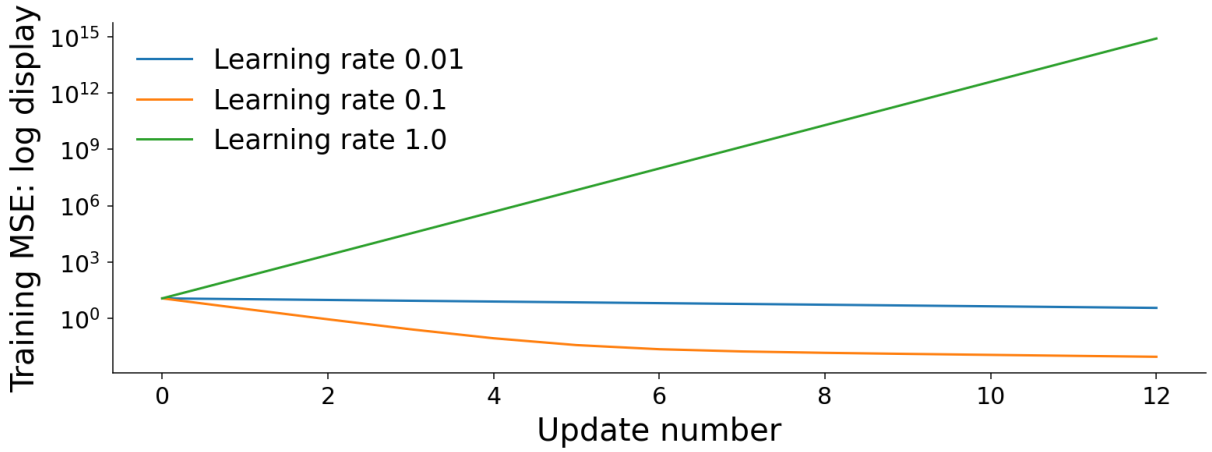
2. Repeat the update with a newly computed gradient. Run 40 iterations on the same three training pairs. The path approaches intercept 1 and slope 2 for this convex quadratic example. Coding Note 6.3.2.

The .1 learning-rate update changes (a,b) from (0,0) to (0.6000,0.8667). Training MSE decreases from 11.6667 to 3.2074. Compute the gradient again at the new parameters before the next update. In this simple convex squared-loss problem, the path approaches the exact fitting line a=1,b=2. The update rule requires an appropriate step size.

2.2 Learning rate changes the size of the update

Definition: The idea
A learning rate controls how far an update moves for a given gradient. A large step can cross a low-loss region and increase loss. A small step can make progress slowly.

Follow the picture. On this graph, a falling line means smaller training error as updates continue. The vertical axis uses a logarithmic scale: equal vertical gaps represent equal multiplication factors. This makes both very large errors and small errors visible in the same picture.



Coding Note, Example 6.4.

Where to use the idea. Refer here when loss rises or oscillates during training. Inspect scaling and learning rate, and select settings using an appropriate validation procedure.

2.2.1 Example 6.4: optional calculation reference

1. Hold the problem fixed and change eta. Compare .01, .1 and 1 for 12 updates. Any loss increase is measured from the same starting parameters and data. Coding Note 6.4.1.

2. Plot training loss on a logarithmic scale. A log scale keeps the divergent path visible with the others. The axis label states that scale explicitly. Coding Note 6.4.2.

Keep the same initial line, observations and gradient calculation. Change only learning rate: .01 gives slower progress, .1 gives effective descent, and 1 produces increasing loss in this numerical example. The figure uses a logarithmic loss axis so the divergent trajectory can be shown beside decreasing trajectories. Training iterations and model depth are separate choices.

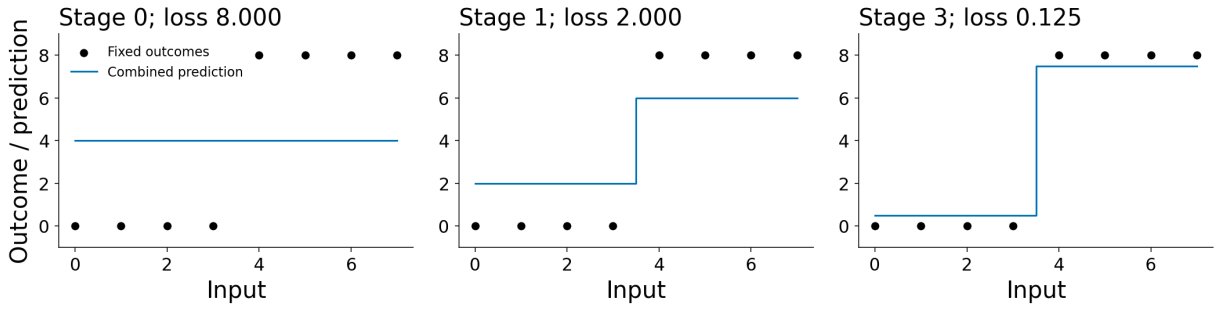
3 Gradient Boosting: Correcting a Prediction Rule

3.1 Boosting adds a correction function

Definition: The idea

Gradient Boosting builds an ensemble by adding fitted correction functions in stages. With half squared-error loss, a residual is the negative gradient with respect to the current prediction, so a shallow tree can be fitted to those residuals.

Follow the picture. Watch the black dots and blue predictions. The black dots are the outcomes we are trying to match; they stay still. The blue predictions begin at four. For outcomes at zero, move the predictions down; for outcomes at eight, move them up. Each small tree supplies another correction based on the errors that remain.



Coding Note, Example 6.5.

Where to use the idea. Refer here to read a boosting implementation: identify the initial prediction, loss, residual/negative-gradient target, correction learner, learning rate and number of stages.

3.1.1 Example 6.5: optional calculation reference

$$F_{m+1}(x) = F_m(x) + \eta T_m(x)$$

1. Create two groups and the initial mean rule. Eight ordered inputs have outcomes 0,0,0,0,8,8,8,8. The initial average forecast is 4 for every training input. Coding Note 6.5.1.

2. Fit a shallow tree to the current residuals. At each stage, fit a one-split tree to actual minus current prediction. The tree learns a correction conditional on the input, then half that correction is added. Coding Note 6.5.2.

3. Compare the prediction stages using fixed axes. Show the same outcomes and rules at stages 0,1,3. Black outcomes remain fixed while blue predictions improve in this constructed training example. Coding Note 6.5.3.

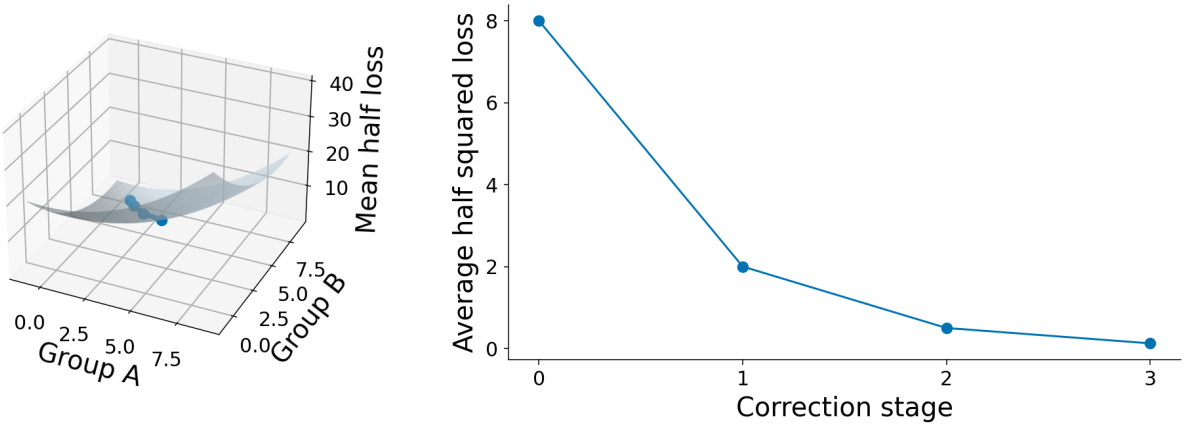
4. Use the library implementation with the same settings. GradientBoostingRegressor uses three one-split trees, learning rate .5 and squared-error loss. Its final predictions match the manual correction loop in this constructed example. Coding Note 6.5.4.

Initial predictions are 4 for every point. The first four residuals are -4 and the last four +4. A one-split tree separates those inputs and predicts the group corrections. Adding half the correction yields predictions 2 and 6. Repeating yields 1 and 7, then .5 and 7.5. Average half squared loss is 8,2,.5,.125 at the four displayed stages. The code fits correction functions in sequence; Random Forest, taught earlier, averages randomized fitted trees.

3.2 Why the correction follows a negative gradient

Definition: The idea
For squared prediction errors, correcting the remaining signed errors takes predictions in a loss-reducing direction. This connects boosting to gradients.

Follow the picture. The three-dimensional diagram now puts the first group's prediction on one floor axis and the second group's prediction on the other. Height still means prediction error. We begin at four for both groups. The observed outcomes are zero and eight, so one prediction needs to fall and the other needs to rise. A correction moves it to two and six, then one and seven. Another correction brings both predictions closer to their observed outcomes. The point moves lower as those errors shrink. For squared error, the remaining signed errors tell each correction tree which way to move the predictions. That is the connection between Gradient Boosting and the downhill gradient idea.



Coding Note, Example 6.6.

Where to use the idea. Refer here to connect the name Gradient Boosting with its loss. Residual fitting is the squared-error instance; other losses supply their own negative-gradient targets.

3.2.1 Example 6.6: optional calculation reference

$$\ell(y, F) = \tfrac{1}{2}(y - F)^2, \qquad -\frac{\partial \ell}{\partial F} = y - F$$

1. Compute the two-coordinate prediction gradient. Use one representative outcome from each of the two equal-sized groups. The negative gradient has coordinates (-4,+4) at the initial predictions (4,4), for the sum of the two half squared losses. Coding Note 6.6.1.

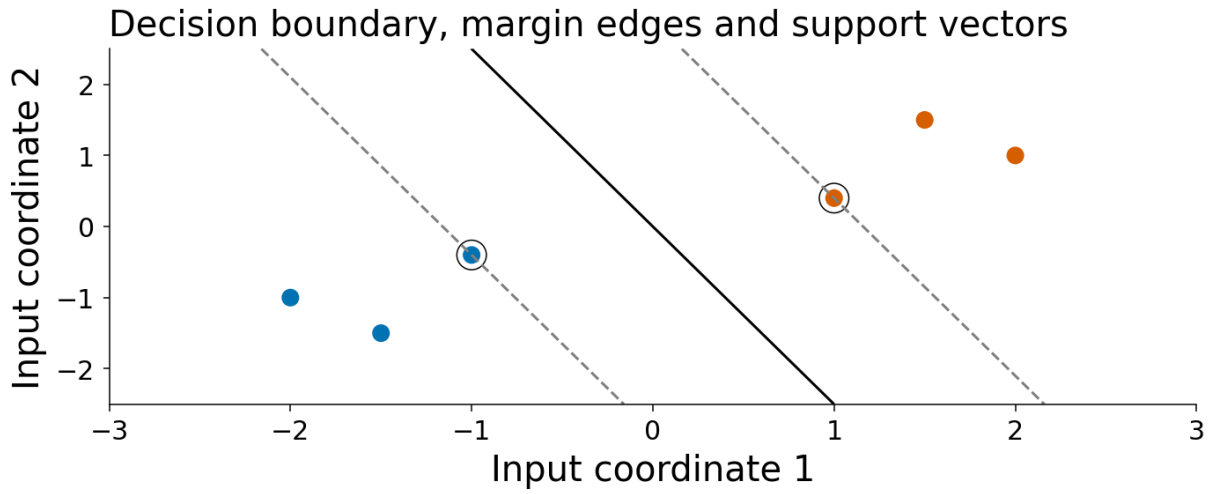
For one observation, half squared-error loss has derivative F-y with respect to its prediction F. Its negative is y-F, the residual used as the next tree target here. The two-coordinate surface uses the two group predictions as horizontal axes, rather than intercept and slope. Its height averages the two half losses. Averaging rescales the surface gradient while preserving the downhill direction. The tree-fitting step supplies a correction function that approximates the desired per-observation changes. Learning rate controls the fraction of that correction added.

4 Support Vectors, Margins and Nonlinear Representations

4.1 SVM learns a separating boundary and a margin

Definition: The idea
A support vector machine (SVM) learns a boundary between categories. Its margin is a strip around that boundary; supporting observations help determine it.

Follow the picture. The margin describes the strip around the dividing line. Imagine widening that strip while keeping the two groups on their appropriate sides. The circled support observations help determine the fitted boundary and its margin. Their positions constrain where the line can go. Real groups can overlap. Fitting can allow some observations inside the strip or on the wrong side, while balancing those errors against the margin. We choose settings by comparing models on held-out observations, using the evaluation ideas from earlier lectures. The visual idea to keep is a boundary, a surrounding strip and supporting points that help determine the fitted rule.



Coding Note, Example 6.7.

Where to use the idea. Refer here when your outcome is a category and you need to understand an SVM boundary, support vectors or the C parameter. Standardize differently scaled inputs inside the training/validation workflow.

4.1.1 Example 6.7: optional calculation reference

$$s(\mathbf{x}) = \mathbf{w} \cdot \mathbf{x} + b$$

1. Fit a linear SVM on a small two-category sample. Construct separated points and fit a linear kernel with C=10. A linear kernel uses ordinary dot-product similarity between the input vectors. The learned score is zero on the decision line and has opposite signs on its two sides. Coding Note 6.7.1.

2. Draw the boundary, margin and supporting observations. Use contours of fitted score -1,0,+1. The circled observations determine the fitted support-vector rule. Coding Note 6.7.2.

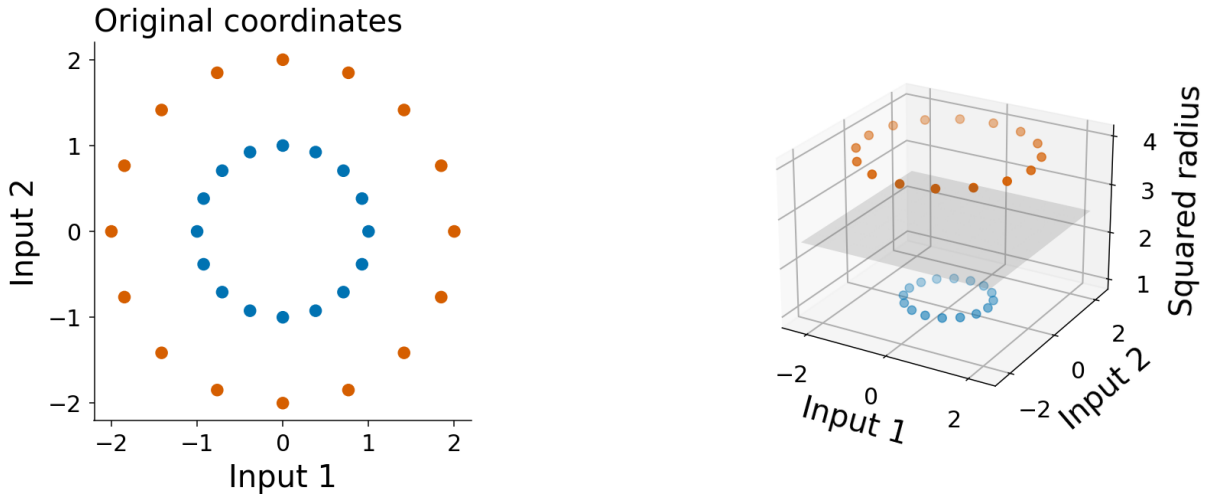
Each point has two measured inputs and one of two category labels. The linear score w dot x plus b is zero on the boundary. Its sign assigns a side. The dashed score lines -1 and +1 mark margin edges for the standard fitted-score normalization. Circled support observations have nonzero coefficients in the fitted rule. In less cleanly separated data, points inside the

margin or misclassified points can also be support vectors. The C setting controls penalties for violations relative to the boundary regularization.

4.2 A feature map can make a curved boundary simpler

Definition: The idea
A feature is an input measurement or a number calculated from measurements. A feature map calculates new inputs that can make a boundary easier to draw.

Follow the picture. Look at two concentric circles of dots. The inner circle is one category and the outer circle is another. A straight line across this flat picture cannot separate the two complete rings. Calculate each dot's distance from the center and square it. Lift each observation by that calculated amount. The outer circle rises higher than the inner circle. A horizontal plane can now separate them. The representation changes, and the separation becomes simpler. A feature map is the rule for calculating those new inputs. This picture illustrates why changing how inputs are represented can help a model capture a curved boundary.



Coding Note, Example 6.8.

Where to use the idea. Refer here when investigating whether a nonlinear decision boundary or engineered features may help. Keep the transformation and its fitted parameters inside the appropriate training workflow.

4.2.1 Example 6.8: optional calculation reference

$$\phi(x_1, x_2) = (x_1, x_2, x_1^2 + x_2^2)$$

1. Construct two circles and the third coordinate. Use 16 observations per circle with radii 1 and 2. The third coordinate separates the groups in height. Coding Note 6.8.1.

2. Display the original circles and lifted plane. Show actual mapped coordinates with fixed axes. The plane separates the constructed transformed observations. Coding Note 6.8.2.

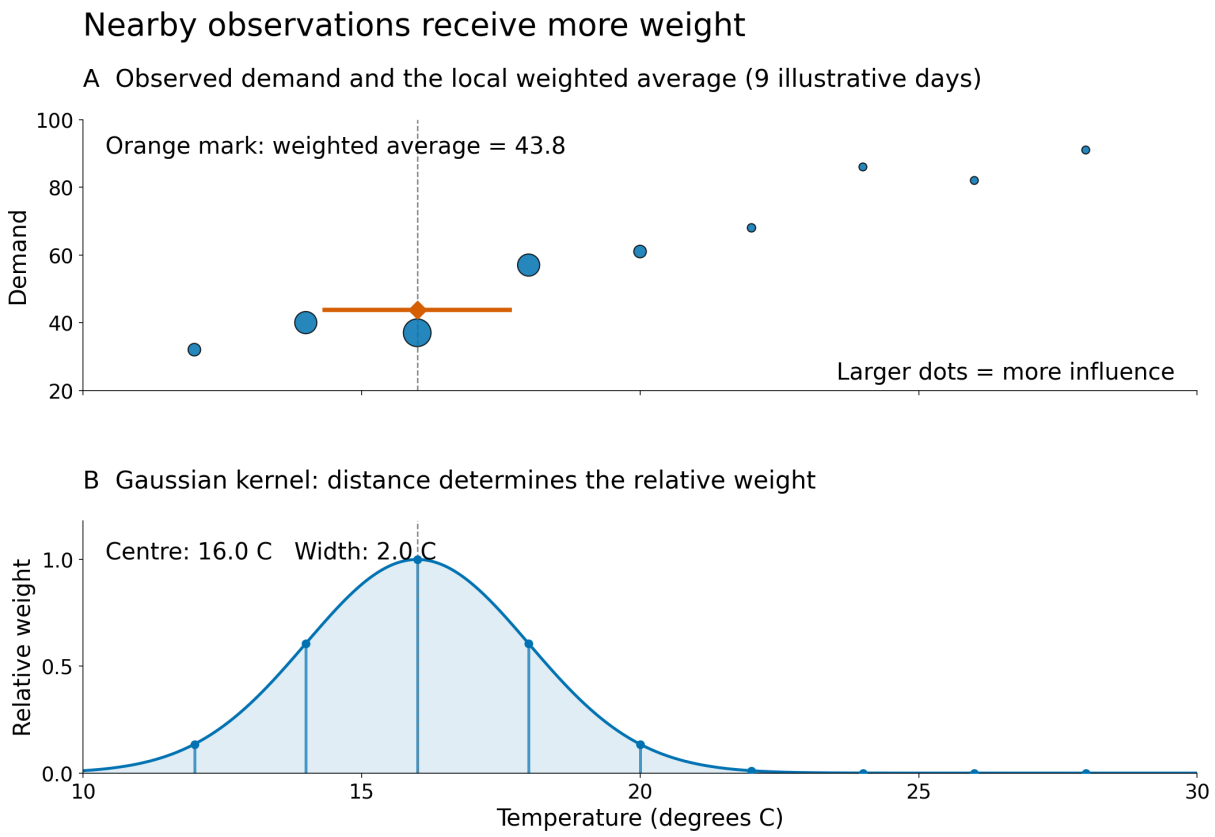
The original sample contains two circles with radii one and two. Squared radius is therefore one or four. Adding that coordinate gives a vertical separation; the fitted separating squared-radius cutoff is 2.50. The same transformation is applied to any new input before using this explicit rule. Kernels offer a way to compute feature-space similarities, sometimes without forming every transformed coordinate. The illustrated radius map defines the transformed coordinates explicitly.

4.3 Gaussian kernel: nearby points have more influence

Definition: The idea

A Gaussian kernel assigns more weight or similarity to nearby inputs and less to distant inputs. Its width controls how quickly influence falls with distance.

Follow the picture. Imagine estimating demand at a temperature of sixteen degrees. Read the two panels together. The upper panel contains observed temperatures and demand. The lower panel shows a Gaussian kernel, a bell-shaped rule for assigning weights around the temperature we care about. Observations near its centre receive larger weights. Their dots above become larger, showing their greater influence. Farther observations receive smaller weights. To form the orange weighted average, multiply each observed demand by its share of the total weight, then add the contributions. Now move the centre to a warmer temperature. The high-weight observations change, and the average changes with them. Keep the centre fixed and widen the bell. More distant observations now contribute more. This width controls how local the averaging is. In a support vector machine, the same Gaussian shape can measure similarity: nearby inputs are more similar. The SVM combines those similarities using coefficients learned during fitting. Our two-panel average makes the distance-to-weight idea visible; the SVM uses that similarity idea to learn its boundary.



Coding Note, Example 6.8A.

Where to use the idea. Refer here when interpreting Gaussian distance weights, width settings and normalized weighted averages. For an SVM, check the trained coefficients and model scaling as well as the kernel setting. The two-panel example uses a local weighted average to explain influence.

4.3.1 Example 6.8A: optional calculation reference

$$k_i = \exp[-(x_i - c)^2/(2h^2)], \quad p_i = k_i / \sum_j k_j, \quad \hat{y}(c) = \sum_i p_i y_i$$

1. Calculate distance weights and their shares. Use nine explicitly constructed days with temperature and demand. Weights sum to one after normalization; the local average remains in the observed demand range. Coding Note 6.8A.1.

2. Connect the upper outcomes and lower kernel weights. Keep both temperature axes aligned; show a centre of 16 and width 2. Bigger dots carry more influence on the orange average. The lower bell gives their raw relative weights. Coding Note 6.8A.2.

At centre 16 and width 2, calculate a Gaussian relative weight from each temperature distance, divide by the total weight and combine the observed demand values. The notebook

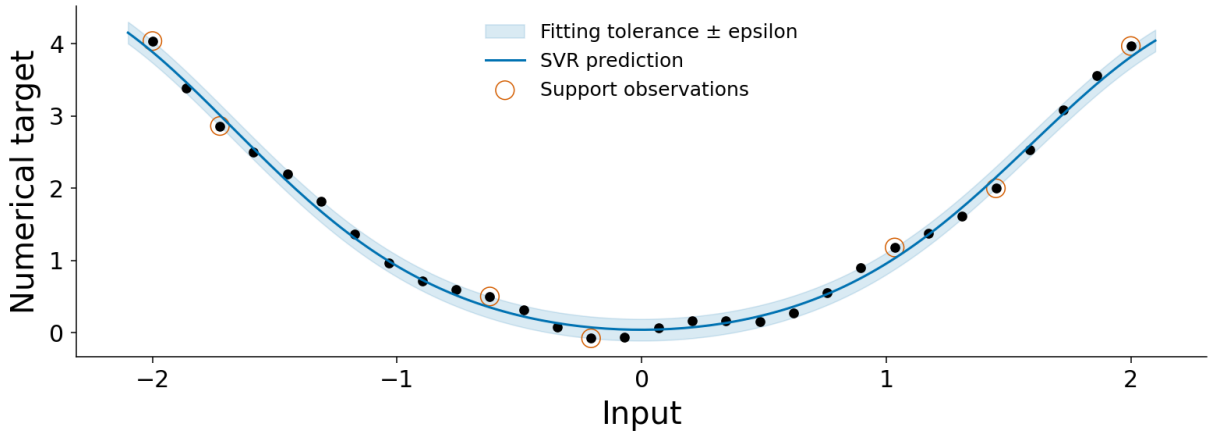
prints every weight and its normalized share. Moving the centre and widening the kernel are separate changes. In an SVM, Gaussian similarities are combined using fitted coefficients; this local-average display explains proximity weights.

4.4 SVR applies support-vector ideas to a numerical target

Definition: The idea

Support Vector Regression (SVR) predicts a number using a fitting rule that allows a chosen error tolerance.

Follow the picture. Forecasting usually asks for a number. Support Vector Regression, or SVR, applies support-vector ideas to that task. Imagine a fitted curve with an allowed miss on each side. Small errors inside that strip receive no error penalty under this fitting rule. Errors beyond the strip contribute a penalty for the part outside it. The strip in this picture shows that chosen tolerance. A curved rule can follow a relationship that bends across the inputs. Our notebook gives one runnable example, including the input preparation and model settings. We still evaluate its predictions on appropriate held-out observations, as in the earlier lectures.



Coding Note, Example 6.9.

Where to use the idea. Refer here for a numerical SVR task. Check input scaling, C, epsilon, kernel settings and training cost; choose them using validation appropriate to your target and information timing.

4.4.1 Example 6.9: optional calculation reference

$$L_\epsilon(y, f) = \max(0, |y - f| - \epsilon)$$

1. Fit a small nonlinear SVR example. Use a deterministic curved target with small constructed variation. Input scaling is fitted in the Pipeline before SVR. Coding Note 6.9.1.

2. Show the fitting tolerance band. Draw prediction plus/minus epsilon and circle support observations. The epsilon band describes the fitting criterion. Coding Note 6.9.2.

The SVR target is a numerical response. For a prediction f and observed target y, first calculate absolute error. Subtract epsilon and keep a positive remainder. An error .10 has zero epsilon-insensitive loss when epsilon=.15; an error .40 has loss .25. The shaded band in the figure expresses this fitting tolerance. Outcome probability intervals require their own uncertainty construction and coverage evaluation. Input standardization is fitted inside the Pipeline. C=10 and epsilon=.15 are stated illustration settings.

5 Neural Networks: Layers, Transformations and Learning

5.1 One neural-network unit computes a weighted transformation

Definition: The idea

A neural-network unit combines input numbers with weights and a bias, then applies an activation function. A weight is an adjustable coefficient; a bias is an added adjustable constant. An activation transforms the combined number; ReLU (rectified linear unit) keeps a positive value and returns zero for a negative value.

Follow the picture. Follow the boxes from left to right. First multiply each input by its weight, then add the contributions and the constant. Next apply the activation rule. Here the rule passes a positive total and changes a negative total to zero. The outgoing arrow carries that resulting number to the next calculation.



Coding Note, Example 6.10.

Where to use the idea. Refer here to read the input/weight shapes of a dense layer, interpret a bias and understand how an activation changes its output.

5.1.1 Example 6.10: optional calculation reference

$$z = w_1 x_1 + w_2 x_2 + b, \qquad h = \max(0, z)$$

1. Calculate a unit by hand and in code. Use the stated two inputs, weights and bias. A unit has an explicit numerical operation that can be inspected. Coding Note 6.10.1.

2. Draw the calculation chain. Each box shows a numerical transformation. An arrow means that a computed value is passed to the next operation. Coding Note 6.10.2.

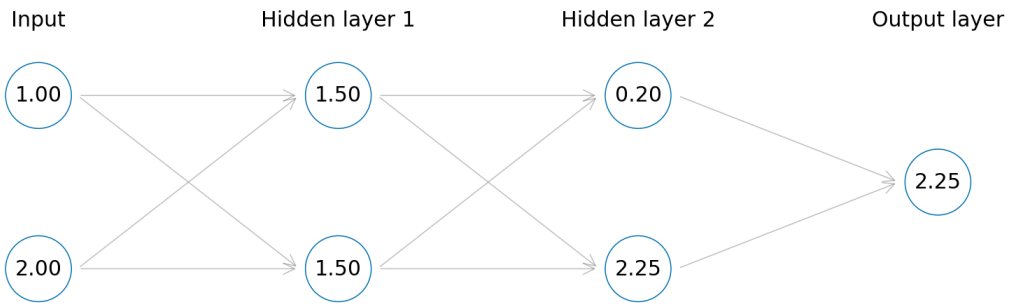
Inputs are 2 and 1. Weights 1 and -1 give products 2 and -1; bias .5 makes the weighted sum 1.5. ReLU passes that positive value as output 1.5. For a negative weighted sum, the same activation returns zero. A unit performs a specific arithmetic transformation. Combining many units allows later stages to use different transformed features.

5.2 Layers compose transformations: a folding analogy

Definition: The idea

A layer is one stage of numerical calculations. Its outputs become the next layer's inputs. Hidden layers are the stages between the original inputs and final output.

Follow the picture. Watch the marked sheet. After one fold, the marks occupy a different arrangement; the next fold works on that new arrangement. A network layer similarly calculates new numbers from the numbers it receives. Those new numbers become the next layer's inputs. In the code, a matrix is simply a table of weights, with one row for each unit's calculation. Training revises those weights while keeping the chosen layer arrangement.



Coding Note, Example 6.11.

Where to use the idea. Refer here to inspect network dimensions and distinguish the number of layers from the number of training updates. The paper fold is a representation analogy, not a simulation of how parameters are fitted.

5.2.1 Example 6.11: optional calculation reference

1. Compute two hidden layers and the output. Use fixed illustrative matrices to keep every intermediate value inspectable. The network has input size 2, hidden sizes 2 and 2, and output size 1. Coding Note 6.11.1.

2. Draw fixed layer values and directed connections. Display two hidden layers and the

output layer. The diagram labels computed values; weights determine how they were combined. Coding Note 6.11.2.

3. Construct the paper-fold analogy. Apply two stated geometric rotations to a marked grid. This is a representation analogy: numerical network layers compose transformations; training adjusts their parameters. Coding Note 6.11.3.

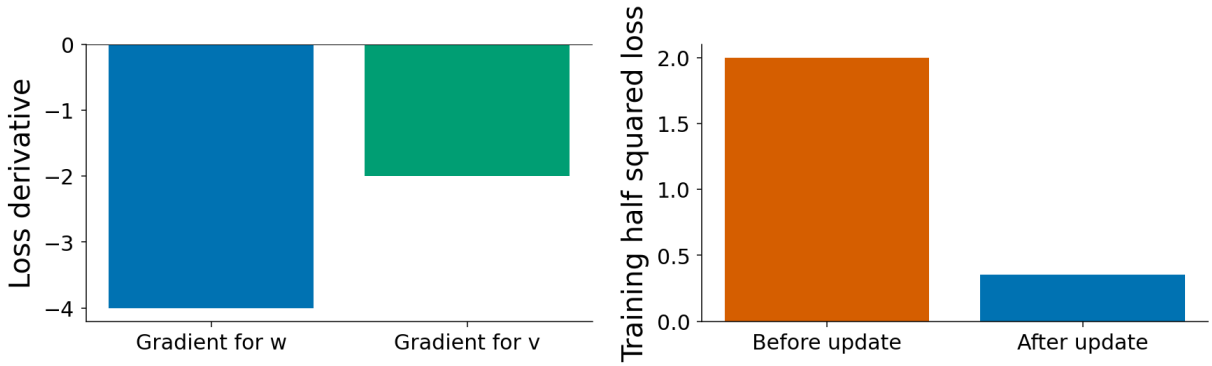
The displayed network computes hidden-layer values [1.5, 1.5] and then [2.25, 0.2]; its final prediction is 2.2500. It has two hidden layers and one parameterized output layer. Input values are shown separately. Imagine deliberate folds in marked paper: each fold changes the arrangement passed to the next fold. The folds represent transformations; parameters determine the particular transformation. An architecture fixes the number of layers, while repeated training updates adjust its parameters. A complex arrangement of folds represents composition, rather than random crumpling until a target appears.

5.3 Backpropagation computes gradients through the layers

Definition: The idea

Backpropagation works backward through a network's calculations to determine how each adjustable weight affects prediction error. Training uses that information to adjust weights.

Follow the picture. Consider a small chain of calculations. Inputs enter, a hidden stage transforms them, and the output gives a prediction. Compare that prediction with the observed target to measure error. Now trace the calculation backward. Ask how changing an output weight would change the error. Continue backward to ask how earlier weights contributed through the later calculations. Backpropagation calculates these effects efficiently through the network. It produces gradients for the adjustable weights. Training can then use those gradients to choose small changes. The arrows running backward carry information about how weights affect error; the next prediction will again be calculated from inputs toward output.



Coding Note, Example 6.12.

Where to use the idea. Refer here to connect a gradient-descent update to neural-network training, inspect derivative signs and separate forward computation, gradient computation and parameter updates.

5.3.1 Example 6.12: optional calculation reference

$$J = \tfrac{1}{2}(\hat{y} - y)^2, \quad h = \max(0, wx), \quad \hat{y} = vh$$

1. Compute the forward values and weight gradients. Use a positive hidden preactivation, whose ReLU slope is one. The output error is -2; local sensitivities transmit it to the two weights. Coding Note 6.12.1.

2. Update both weights using the old gradients. Take one simultaneous update with learning rate .1. New loss is .3528 in this explicit example. Coding Note 6.12.2.

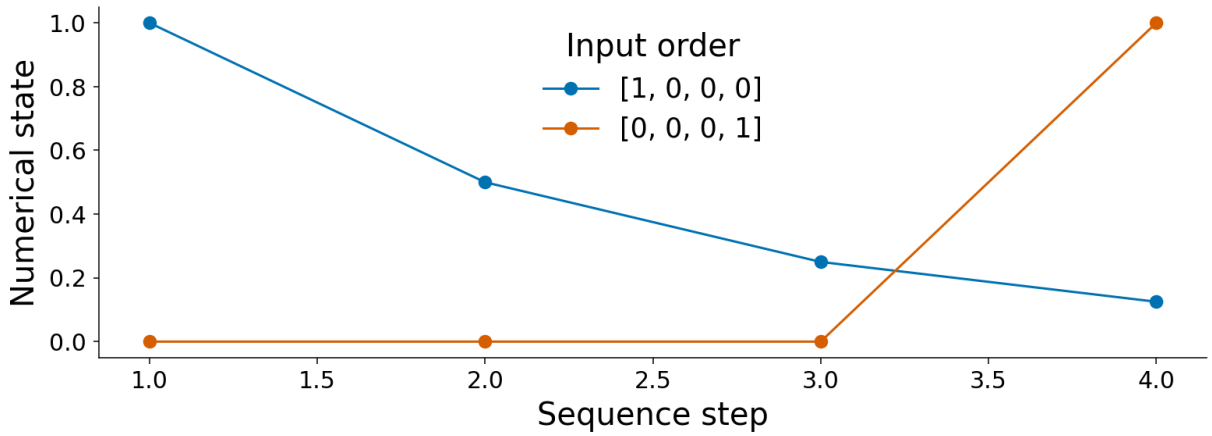
The chain example has input 2, hidden weight .5 and output weight 1. The hidden value is 1, prediction is 1 and target is 3. Half squared loss equals 2. The loss derivative with respect to the output prediction is -2. Multiplying by hidden value 1 gives output-weight gradient -2. Continuing through output weight 1, the active ReLU slope 1 and input 2 gives hidden-weight gradient -4. A simultaneous .1 update changes the weights to .9 and 1.2; the new prediction is 2.16 and loss .3528. Backpropagation computes these derivatives through the saved forward calculations; the optimizer uses them for updates.

6 Ordered Inputs and LSTM Memory

6.1 Sequence order can matter to a prediction

Definition: The idea
A sequence is an ordered set of observations. A state is a numerical summary carried from one step to the next. A recurrent computation updates that state using the preceding state and the current input.

Follow the picture. Imagine demand jumped four days ago, compared with demand jumping today. The average of the four days can match, while the recent situation differs. Watch how an early input gradually fades in the running summary. A late input remains larger at the end because fewer update steps have reduced it.



Coding Note, Example 6.13.

Where to use the idea. Refer here when a feature summary discards timing that may matter, such as the order of recent demand observations. Define the information cutoff and target before choosing a sequence model.

6.1.1 Example 6.13: optional calculation reference

$$s_t = 0.5 s_{t-1} + x_t$$

1. Process the same numbers in different orders. Start at zero state and update through each sequence. Different final states reveal that the computation retains order. Coding Note 6.13.1.

2. Compare the numerical memories. Use the same axes and update rule. Only input order changes. Coding Note 6.13.2.

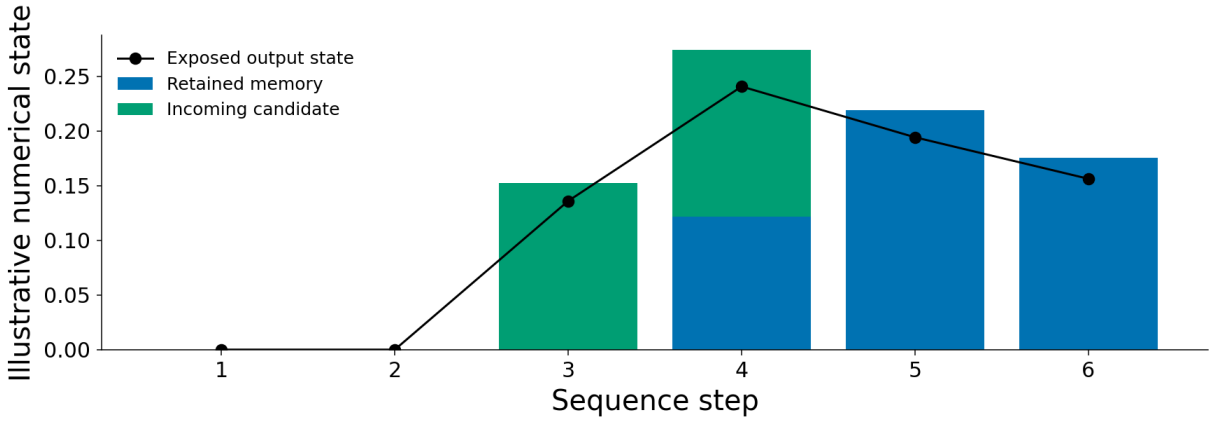
The two input orders have equal means .25. With initial state zero and update s_new=.5 s_old+x, an input of one at the first step gradually fades to .125 by step four. An input of one at the last step leaves final state one. A recurrent state therefore captures an ordered history under a specified update rule. A sequence model uses this kind of evolving representation when learning a forecast mapping.

6.2 LSTM controls retained, incoming and output information

Definition: The idea
Long Short-Term Memory (LSTM) is a neural network for ordered inputs. It carries numerical memory between observations and learns how much old and new information to retain.

Follow the picture. An LSTM, short for Long Short-Term Memory, is a neural network that carries a memory cell through ordered observations. Imagine a running numerical note about

recent demand. A gate is a calculated multiplier between zero and one. Point eight keeps eighty percent of the quantity it multiplies. A forget gate controls how much old information to retain. An input gate controls how much new information to add. An output gate controls what is passed onward from the updated memory. Training learns how to calculate these multipliers from the sequence. This gives a network a way to handle information that remains useful across several dates.



Coding Note, Example 6.14.

Where to use the idea. Refer here to distinguish cell memory from exposed output and to inspect input/sequence/state shapes before reading a library LSTM implementation. Evaluate future predictions chronologically and compare with simple alternatives.

6.2.1 Example 6.14: optional calculation reference

$$c_t = f_t c_{t-1} + i_t g_t, \qquad h_t = o_t \tanh(c_t)$$

1. Specify gates and candidate transformation. Use candidate inputs 0,0,1,1,0,0 and initial cell memory zero. Constant gates correspond to a valid fixed choice of gate weights and biases. Coding Note 6.14.1.

2. Display the cell-state contributions and exposed output. Plot retained and incoming memory contributions separately. The black output curve applies the output gate and tanh to the resulting memory. Coding Note 6.14.2.

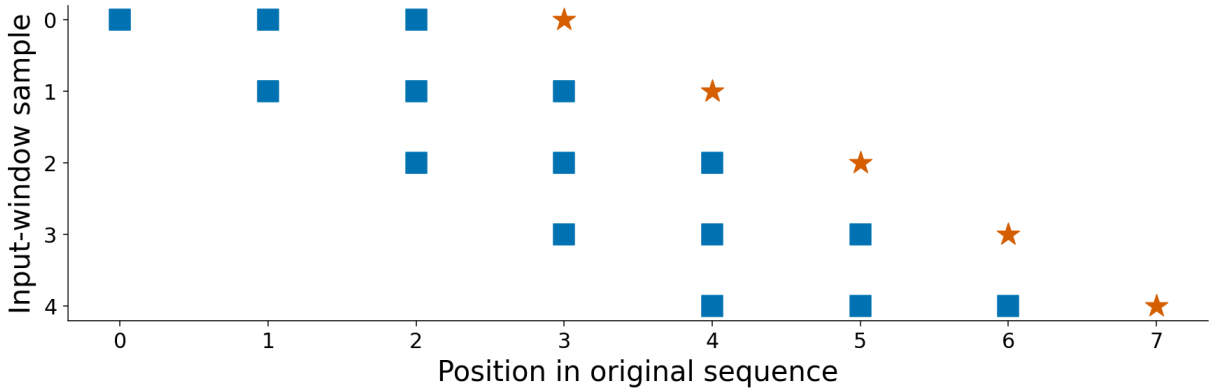
The example fixes the forget gate at .8, input gate at .2 and output gate at .9. Candidate input is tanh(x). At the first nonzero input, tanh(1) is about .7616. Incoming memory is .2 times .7616, about .1523. The next memory combines retained prior memory and this incoming contribution. The exposed state is .9 times tanh(updated memory). These values are computed by an LSTM cell with stated fixed gate settings. In training, gate and candidate transformations have learned weights and biases. Sequence length, network architecture and evaluation remain explicit modeling choices.

6.3 Prepare a time-series window for a sequence model

Definition: The idea

A sequence input window groups the most recent observations into an ordered input for a later target. In a common LSTM interface, input dimensions are samples, time steps and features.

Follow the picture. Read the first row of the window picture: ten, twelve and eleven are the three earlier observations; fourteen is the later value to predict. Shift one place right: twelve, eleven and fourteen now predict thirteen. Each row is one sample. Three positions within a row are three time steps. One number at each position means one feature. This is why the array has five rows of examples, three ordered positions and one feature per position.



Coding Note, Example 6.15.

Where to use the idea. Refer here when reshaping a chronological dataset for an LSTM. Specify target, origin, horizon and known input variables; fit preprocessing on training data, select architecture using validation and evaluate later outcomes. These illustrations add no project model requirements.

6.3.1 Example 6.15: optional calculation reference

1. Construct ordered windows and subsequent targets. Use values 10,12,11,14,13,15,16,15 with three-step inputs. Each target occurs immediately after its input window. Coding Note 6.15.1.

2. Draw the windows on the original observation order. Input positions and targets use different markers. Each row has three earlier inputs and one later target. Coding Note 6.15.2.

Three ordered observations form each input window. With eight observations, the five next targets are 14,13,15,16,15. The array shape (5,3,1) means five samples, three time steps and one feature per step. Later models may have several measured features at each time step. Define which features are known at the forecasting origin, split targets chronologically and

fit preprocessing within the training period before applying the learned model.

7 Begin and Complete Your Selected Final Project

7.1 Choose one of the three existing cases

The project was released in Week 0, before Lecture 1. Choose one supplied case that fits your interests and complete its tasks:

1. P1: healthcare. Predict patient disease progression from the clinical data. The observation unit is a patient; evaluate on held-out patients following the current notebook.

2. P2: finance and banking. Predict the specified profitability ratio from the bank snapshot. Evaluate on held-out banks; describe the contemporaneous inputs and the net-income-to-ending-assets ratio used by this case.

3. P3: industry and business operations. Forecast next-day bike-sharing demand. Use inputs available at the preceding origin and chronological evaluation. The operating context is fleet allocation and staffing.

7.2 Complete the tasks in the selected notebook

Complete data preparation and exploratory analysis; OLS, Ridge, Lasso and Random Forest; held-out metrics and a comparison table; coefficient and feature-importance interpretation; domain recommendations and the supplied discussion prompts. Keep transformations and model selection within the appropriate training and validation samples. Report actual code outputs, units and their interpretation.

Prepare the completed notebook with code, requested outputs and written explanations. Include the accompanying files required by that case so the work can be run and understood. Use the numbered Coding Notes as references for the relevant operations.

7.3 Deadline and Blackboard submission

The current syllabus deadline is Sunday, May 2, 2027 at 11:59 PM Central Daylight Time (America/Chicago). This is the seventh teaching week, called Week 6 when numbered Week 0–6. The instructor will open a Blackboard project submission link. Submit your selected completed project through that link. Use the displayed calendar date when planning your work.

7.4 Completion credit and correct-step credit

The final project is worth 200 points. Half of each required item's points reward a substantive attempt: actual code or analysis addressing that task. Substantively attempting every required item therefore earns at least 100/200. The remaining half rewards correct steps, completeness, interpretation and quality. Correct intermediate work earns partial credit even

when a later step or final answer has an error. A heading or unchanged supplied template does not demonstrate an attempt.

Category	Attempt	Quality	Total
Preparation and exploratory analysis	15	15	30
Model implementation	35	35	70
Evaluation and comparison	20	20	40
Interpretation and domain analysis	20	20	40
Conclusions and recommendations	10	10	20
Total	100	100	200

The existing task-specific rubric criteria describe correctness and quality within those categories. Partly attempted work receives completion credit for the corresponding items. Lecture 6 assigns no additional homework; use this stage to work through and finish your chosen project.